

ÁRBOLES DE DECISIÓN Y MÉTODOS DE ENSAMBLE

Guía Maestra de Estudio — Handbook Técnico Profesional

Análisis Predictivo de Datos · Machine Learning · Período 6

Emanuel Cabrera Novoa · 2026

Docente: Jhon Quiza

Tabla de Contenidos

Cap.	Tema	Contenido Principal
1	Visión General del Proyecto	Objetivos, tecnologías, arquitectura
2	Mapa Global de Temas	Jerarquía completa de conceptos
3	Árboles de Decisión — Teoría	Estructura, criterios, fórmulas
4	Árboles — Implementación	Clasificación, regresión, podado, pipelines
5	Métodos de Ensamble — Teoría	Bias-varianza, clasificadores débiles
6	Bagging y Random Forest	Bootstrap, OOB, decorrelación, código
7	Boosting: AdaBoost	Algoritmo, fórmulas, implementación
8	Boosting: Gradient Boosting	Residuos, gradientes, hiperparámetros
9	Boosting Avanzado	XGBoost, LightGBM, CatBoost
10	Conceptos Fundamentales	Los 15 conceptos más críticos
11	Relaciones Entre Conceptos	Mapa de interacciones
12	Guía de Estudio Definitiva	Roadmap, prerequisites, estrategia
13	Cheatsheet & Fórmulas	Referencia rápida con todo
14	Ejemplos de Código Completos	Básicos, intermedios, avanzados
15	Conclusiones y Recomendaciones	Síntesis y próximos pasos

Capítulo 1 — Visión General del Proyecto

Este material corresponde al módulo de **Árboles de Decisión y Métodos de Ensamble** del curso de Análisis Predictivo de Datos (Machine Learning, Período 6), impartido por el docente **Jhon Quiza**. El objetivo central es dominar la familia completa de modelos basados en árboles, desde el árbol individual hasta los métodos de ensamble de última generación.

Archivos Analizados

Archivo	Tipo	Contenido Central
Árboles de decisión.pdf	Presentación (20 diapositivas)	Teoría: estructura, entropía, Gini, ventajas/desventajas
Métodos de ensamble.pdf	Presentación (11 diapositivas)	Bagging, bootstrap, random forest — teoría matemática
Métodos de boosting.pdf	Presentación (15 diapositivas)	AdaBoost, Gradient Boosting — fórmulas completas
Modelos_de_árboles_de_decisión.ipynb	Notebook	Clasificador árbol: Adult dataset, GridSearchCV, podado
Modelos_de_árboles_de_decisión_regresión.ipynb	Notebook	Regresor árbol: Auto-MPG, pipelines, scaling
Bagging_y_random_forest.ipynb	Notebook	BaggingRegressor, RandomForestRegressor, OOB
Modelos_de_boosting.ipynb	Notebook	AdaBoost vs GradientBoosting vs RandomForest
Modelos_de_boosting_avanzados.ipynb	Notebook	XGBoost, LightGBM, CatBoost — comparativa completa

Tecnologías y Librerías

Librería	Versión aprox.	Uso en el curso
scikit-learn	≥1.4	DecisionTree, RandomForest, AdaBoost, GradientBoosting, GridSearchCV, Pipeline
XGBoost	3.2.0	XGBClassifier con soporte nativo de categóricas y early stopping
LightGBM	4.6.0	LGBMClassifier con GOSS, EFB, leaf-wise growth
CatBoost	1.2.10	CatBoostClassifier con Ordered Target Statistics
pandas	≥2.0	Manipulación de DataFrames, EDA
numpy	≥2.0	Operaciones numéricas, logspace para hiperparámetros
matplotlib	≥3.8	Visualización de árboles con plot_tree
scipy	≥1.10	Distribuciones estadísticas para RandomizedSearchCV

Arquitectura General del Aprendizaje

El curso sigue una progresión pedagógica deliberada:

Nivel	Modelo	Objetivo Pedagógico
Base	Árbol de decisión individual	Entender el mecanismo de partición y criterios de impureza
Nivel 1	Bagging / Random Forest	Reducir varianza promediando modelos independientes

Nivel 2	AdaBoost	Reducir sesgo re-pesando muestras difíciles secuencialmente
Nivel 3	Gradient Boosting	Optimizar función de pérdida ajustando residuos via gradiente
Nivel 4	XGBoost / LightGBM / CatBoost	Escalar GB con regularización, histogramas y soporte nativo

Capítulo 2 — Mapa Global de Temas

ÁRBOLES DE DECISIÓN Y MÉTODOS DE ENSAMBLE

■

■ ■ ■ 1. ÁRBOLES DE DECISIÓN

■ ■ ■ ■ 1.1 Estructura

- ■ ■ ■ ■ Nodo raíz (Root node)
- ■ ■ ■ ■ Nodos interiores (Internal nodes)
- ■ ■ ■ ■ Arcos / Ramas (Branches)
- ■ ■ ■ ■ Hojas / Nodos terminales (Leaf nodes)

■ ■ ■ ■ 1.2 Criterios de impureza (clasificación)

- ■ ■ ■ ■ Entropía $H(S) = -\sum p(c) \log p(c)$
- ■ ■ ■ ■ Ganancia de información $GI(S,a)$
- ■ ■ ■ ■ Índice de Gini $= 1 - \sum p_i^2$

■ ■ ■ ■ 1.3 Criterios de partición (regresión)

- ■ ■ ■ ■ MSE (Error cuadrático medio)
- ■ ■ ■ ■ MAE (Error absoluto medio)
- ■ ■ ■ ■ Desviación de Poisson

■ ■ ■ ■ 1.4 Regularización / Podado (Pruning)

- ■ ■ ■ ■ Post-poda: `ccp_alpha` (Cost-Complexity Pruning)
- ■ ■ ■ ■ Pre-poda: `max_depth`, `min_samples_split`, `min_samples_leaf`

■ ■ ■ ■ 1.5 Características especiales

- ■ ■ ■ ■ Modelo caja blanca (white-box)
- ■ ■ ■ ■ Importancia de características (`feature_importances_`)
- ■ ■ ■ ■ Invariante al escalado
- ■ ■ ■ ■ Tolerante a datos nulos

■ ■ ■ ■ 1.6 Sobreajuste (Overfitting)

■

■ ■ ■ 2. MÉTODOS DE ENSAMBLE

■ ■ ■ ■ 2.1 Concepto: clasificadores débiles $\rightarrow$ fuertes

- ■ ■ ■ ■ $f_{ens}(x) = \sum \alpha_i f_{i}(x)$

■ ■ ■ ■ 2.2 Taxonomía por lo que reducen

- ■ ■ ■ ■ BAGGING $\rightarrow$ reduce varianza
- ■ ■ ■ ■ BOOSTING $\rightarrow$ reduce sesgo

■

■ ■ ■ 3. BAGGING Y RANDOM FOREST

■ ■ ■ ■ 3.1 Bootstrap (remuestreo con reemplazo)

■ ■ ■ ■ 3.2 Bagging: $f_{bag}(x) = (1/M) \sum f_{i}(x)$

- ■ ■ ■ ■ Out-of-Bag (OOB) validation
- ■ ■ ■ ■ Modelo caja negra

■ ■ ■ ■ 3.3 Random Forest

- ■ ■ ■ ■ Decorrelación por subconjunto aleatorio de m features
- ■ ■ ■ ■ Regresión: promedio de árboles
- ■ ■ ■ ■ Clasificación: voto mayoritario
- ■ ■ ■ ■ Valores típicos: $m = \sqrt{p} \text{ (clas.)} \cdot p/3 \text{ (reg.)}$

■

■ ■ ■ 4. BOOSTING

■ ■ ■ ■ 4.1 AdaBoost (Freund & Schapire, 1996)

- ■ ■ ■ ■ Re-pesar muestras mal clasificadas
- ■ ■ ■ ■ $\alpha_i = \frac{1}{2} \ln((1-\epsilon_i)/\epsilon_i)$
- ■ ■ ■ ■ $\hat{f} = \text{sign}(\sum \alpha_i h_i(x))$

■ ■ ■ ■ 4.2 Gradient Boosting

- ■ ■ ■ ■ Optimización de función de pérdida
- ■ ■ ■ ■ Residuos = gradiente negativo de L
- ■ ■ ■ ■ $F(x) = F_{init}(x) + \eta \cdot h(x)$

■

■ ■ ■ 5. BOOSTING AVANZADO

■ ■ ■ ■ 5.1 XGBoost

- ■ ■ ■ ■ Regularización L1/L2 explícita

- ■■■ Newton Boosting (gradiente 2do orden)
- ■■■ Búsqueda de splits con cuantiles ponderados
- 5.2 LightGBM
 - ■■■ Histogramas en lugar de búsqueda exacta
 - ■■■ Leaf-wise growth (vs level-wise)
 - ■■■ GOSS: muestreo por gradientes
 - ■■■ EFB: fusión de features dispersas
- 5.3 CatBoost
 - Ordered Target Statistics (sin target leakage)
 - Combinaciones automáticas de categóricas
 - Árboles simétricos para predicción rápida

Capítulo 3 — Árboles de Decisión: Teoría Completa

3.1 ¿Qué es un Árbol de Decisión?

Un árbol de decisión es una estructura jerárquica que divide el espacio de características mediante reglas del tipo '**si-entonces**'. Es una analogía directa al razonamiento humano: dado un conjunto de preguntas con respuestas binarias o categóricas, seguimos la cadena de respuestas hasta llegar a una decisión final.

La intuición: si quieres comprar un auto usado, preguntas: ¿Es de marca reconocida? → Sí → ¿Tiene más de 100.000 km? → No → ¿El vendedor es el propietario? ... hasta llegar a la decisión COMPRAR / NO COMPRAR.

Elementos de un Árbol de Decisión

Elemento	Descripción	Analogía
Nodo raíz (Root)	Primera pregunta / característica más importante	La pregunta más discriminativa del árbol
Nodos interiores (Internal)	Preguntas intermedias sobre otras características	Condiciones anidadas en el proceso de decisión
Arcos / Ramas (Branches)	Posibles valores que toma cada característica	Las respuestas posibles a cada pregunta
Hojas (Leaf nodes)	Decisiones finales / predicciones del modelo	El resultado: clase o valor numérico

3.2 ¿Cómo se Construye un Árbol? El Algoritmo CART

El algoritmo **CART (Classification and Regression Trees)** construye árboles de forma **greedy (voraz)** y **top-down**: en cada nodo, busca exhaustivamente el atributo y el punto de corte que mejor divida los datos según un criterio de impureza. Este proceso se repite recursivamente hasta alcanzar un criterio de parada.

El pseudoalgoritmo es:

Criterio de Parada (Base Cases)

- El nodo es puro: todos los datos pertenecen a la misma clase.
- Se alcanzó la profundidad máxima (`max_depth`).
- El nodo tiene menos muestras que `min_samples_split`.
- La ganancia de información es cero o inferior a un umbral.

3.3 La Entropía — Criterio de Impureza

La **entropía** proviene de la teoría de la información (Shannon, 1948). Mide la *incertidumbre* o *impureza* de un conjunto de datos: cuánta información necesitamos para describir correctamente la clase de un elemento aleatorio del conjunto.

Fórmula de la Entropía

$$H(S) = -\sum_{c \in C} p(c) \cdot \log_2 p(c)$$

Donde: **S** = conjunto de datos, **C** = conjunto de clases, **p(c)** = proporción de muestras de clase *c* en *S*.

Propiedades e Intuición de la Entropía

- **H(S) = 0** → Nodo completamente puro. Todos los datos son de la misma clase. Máxima certeza.
- **H(S) = 1** → Máxima impureza (para 2 clases). Los datos están 50/50 entre clases. Máxima incertidumbre.
- La entropía siempre está en $[0, \log_2(|C|)]$. Para 2 clases: $[0, 1]$. Para *k* clases: $[0, \log_2(k)]$.
- El árbol selecciona el **atributo con MENOR entropía** tras la división (equivalente al de MAYOR ganancia de información).

Ejemplo numérico: Dataset de Tenis

Con 7 días: 4 días se jugó tenis (Yes), 3 no se jugó (No). La entropía del conjunto raíz es:

3.4 Ganancia de Información

La **ganancia de información** mide cuánto reduce la entropía al dividir por un atributo determinado. Es la diferencia entre la entropía del padre y la entropía ponderada de los hijos:

Ganancia de Información

$$GI(S, a) = H(S) - \sum_{v \in \text{values}(a)} |S_v|/|S| \cdot H(S_v)$$

Interpretación: Si al dividir por el atributo 'a', los subconjuntos resultantes son más puros que el conjunto original, la ganancia es positiva. **Se elige el atributo con la mayor ganancia de información.**

3.5 Índice de Impureza de Gini

El índice de Gini es la probabilidad de clasificar incorrectamente una muestra aleatoria si se etiquetara según la distribución de clases del nodo. Es computacionalmente más eficiente que la entropía (no requiere logaritmos).

Impureza de Gini

$$Gini(S) = 1 - \sum_i (p_i)^2$$

$$Gini_split = |L|/(|L|+|R|) \cdot Gini(L) + |R|/(|L|+|R|) \cdot Gini(R)$$

Ejemplo de Cálculo de Gini

Entropía vs Gini: ¿Cuándo usar cada uno?

Criterio	Entropía	Gini
Cálculo	Requiere logaritmos (más costoso)	Solo potencias (más rápido)
Sensibilidad	Favorece nodos muy puros aunque pequeños	Favorece divisiones de igual tamaño
Resultado práctico	Casi siempre igual que Gini	Default en scikit-learn
Usar cuando...	Se necesita el árbol más informativo posible	Se prioriza velocidad computacional

3.6 Criterios para Regresión

En tareas de regresión, el árbol no busca pureza de clase sino minimizar el error de predicción en cada nodo:

Criterio	Fórmula	Cuándo usarlo
MSE (default)	$MSE = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$	Cuando los errores grandes deben penalizarse más
MAE	$MAE = (1/n) \cdot \sum y_i - \hat{y}_i $	Cuando hay outliers que no deben dominar
Friedman MSE	MSE con corrección Friedman	Boosting (usa potencial de mejora)
Poisson	$-\sum y_i \cdot \log(\hat{y}_i) + \hat{y}_i$	Variables de conteo (eventos por unidad de tiempo)

3.7 Ventajas y Desventajas de los Árboles de Decisión

Ventajas	Desventajas
Fáciles de entender e interpretar (caja blanca)	Tendencia al sobreajuste sin regularización
Capturan relaciones no lineales	Alta varianza: sensibles a pequeñas variaciones en datos
Aprendizaje rápido	Predicciones discontinuas en regresión (constantes por hoja)
Selección implícita de características	No son los más precisos en solitario
No requieren manejo de datos nulos	Pueden crear fronteras de decisión muy complejas
No requieren escalado de variables	Difíciles de interpretar cuando son muy profundos
Importancia de características disponible	Algoritmo greedy → no garantiza árbol óptimo global

Capítulo 4 — Árboles de Decisión: Implementación Completa

4.1 Clasificador con Árbol de Decisión

Dataset: **Adult Income** (UCI), 32.561 personas, tarea: predecir si ingresos son $\leq 50K$ o $> 50K$. El árbol sin regularización sobreajusta completamente (accuracy 100% en train, 77% en test).

Pipeline Completo: Carga, Partición y Entrenamiento

4.2 Podado del Árbol (Pruning)

El podado es el proceso de reducir la complejidad del árbol para mejorar su generalización. Existen dos estrategias principales, que se aplican después de entrenar el árbol completo (post-poda) o durante el entrenamiento (pre-poda).

4.2.1 Post-poda: Factor de Complejidad `ccp_alpha`

El parámetro **`ccp_alpha`** (Cost-Complexity Pruning Alpha) determina un umbral de costo a partir del cual se podan ramas. Cuanto mayor sea `ccp_alpha`, más ramas se podan, resultando en un árbol más simple. El algoritmo poda primero los 'enlaces más débiles' (nodos que menos contribuyen a la reducción del error).

4.2.2 Pre-poda: Límites Durante la Construcción

En la pre-poda se establecen criterios de parada *antes* de que el árbol llegue a ser demasiado complejo. Es más eficiente computacionalmente que la post-poda.

Método	ccp_alpha (Post-poda)	max_depth / min_samples_leaf (Pre-poda)
¿Cuándo se aplica?	Después de construir el árbol completo	Durante la construcción del árbol
Ventaja	Encuentra el árbol óptimo de manera sistemática	Más rápido computacionalmente
Desventaja	Más costoso (construye árbol completo primero)	Puede ser subóptimo si el criterio es demasiado estricto
Resultado	Árbol podado global y coherente	Árbol que nunca creció más allá del criterio
Uso recomendado	Cuando se tiene tiempo y se quiere interpretabilidad	Cuando el dataset es grande o se necesita rapidez

4.3 Árbol de Regresión

Dataset: **Auto-MPG**, 398 vehículos. Tarea: predecir el consumo de combustible (mpg). Característica clave demostrada: **los árboles funcionan con datos nulos** (no requieren imputación previa).

¿Por qué el Escalado No Ayuda a los Árboles?

Esta es una de las propiedades más importantes e incomprendidas de los árboles:

- **Invarianza al escalado:** Los árboles basan sus decisiones en umbrales. Si escalamos, el umbral se ajusta proporcionalmente y el resultado es idéntico.
- **Importancia relativa:** Lo que importa es la relación entre características, no su escala absoluta. Una característica con mayor varianza es simplemente más importante, no 'más grande'.
- **Demostrado empíricamente:** En el notebook, aplicar StandardScaler + ccp_alpha al árbol Auto-MPG produce el mismo RMSE (3.54) que sin escalar.

NOTA: Aplicar StandardScaler a un árbol no hace daño, pero tampoco ayuda. Solo agrega complejidad innecesaria al pipeline.

Cuándo Sí tiene sentido escalar con árboles

- Cuando se **compara** con modelos que sí requieren escalado (redes neuronales, SVM, modelos lineales).
- Como paso previo a **reducción de dimensionalidad** (PCA, etc.) antes del árbol.
- En **algoritmos híbridos** donde el árbol es un componente dentro de un pipeline más complejo.

Capítulo 5 — Métodos de Ensamble: Teoría

5.1 ¿Qué son los Métodos de Ensamble?

Los métodos de ensamble combinan múltiples **clasificadores o regresores débiles** (weak learners) para formar un predictor más robusto y preciso. Un clasificador débil es aquel cuya capacidad de predicción es solo ligeramente mejor que una apuesta aleatoria.

Predictor de Ensamble

$$f_{\text{ens}}(x) = \sum_{m=1}^M \alpha_m \cdot f_m(x)$$

donde $f_m(x)$ es el m -ésimo estimador entrenado en $D_m \subset D$

5.2 El Trade-off Sesgo-Varianza

El error esperado de un modelo puede descomponerse en tres términos fundamentales. Esta descomposición justifica matemáticamente por qué existen dos familias de métodos de ensamble:

Descomposición Sesgo-Varianza-Ruido

$$E[(y - f(x))^2] = [f(x) - E(f(x))]^2 + E[(f(x) - E(f(x)))^2] + E[\epsilon^2]$$

Error Total = Sesgo² + Varianza + Ruido irreducible

Componente	Definición	Causa típica	Cómo reducir
Sesgo (Bias)	Error sistemático: cuánto se equivoca el modelo de forma consistente.	Modelos demasiado simples (underfitting).	Modelos más complejos, Boosting
Varianza	Sensibilidad a variaciones en los datos de entrenamiento.	Modelos demasiado complejos (overfitting).	Modelos más simples, regularización, Bagging
Ruido irreducible	Variabilidad inherente de los datos.	Datos ruidosos, variables faltantes.	No se puede reducir con el modelo

Analogía Intuitiva del Sesgo-Varianza

Imagina que eres un arquero apuntando al centro de un blanco:

- **Alto sesgo, baja varianza:** Todas las flechas caen en el mismo lugar, pero lejos del centro. El modelo es consistente pero sistemáticamente incorrecto.
- **Bajo sesgo, alta varianza:** Las flechas están alrededor del centro, pero muy dispersas. El modelo acierta en promedio pero es impredecible.
- **Bajo sesgo, baja varianza:** Todas las flechas cerca del centro. El ideal que buscamos.
- **Alto sesgo, alta varianza:** El peor caso. Las flechas están dispersas y lejos del centro.

5.3 Taxonomía de Métodos de Ensamble

Característica	BAGGING	BOOSTING
Objetivo	Reducir VARIANZA	Reducir SESGO
Entrenamiento	Paralelo (independiente)	Secuencial (cada modelo corrige al anterior)
Datos usados	Bootstrap: muestras con reemplazo	Los mismos datos, re-pesados o con residuos
Combinación	Promedio simple o voto mayoritario	Suma ponderada (pesos por precisión)
Riesgo principal	No reduce sesgo alto	Puede sobreajustarse con muchas iteraciones
Ejemplos	Bagging, Random Forest	AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost
Paralelizable	Sí (fácilmente)	No (por naturaleza secuencial)
Interpretabilidad	Caja negra (aunque tiene feature importance)	Caja negra (más complejo aún)

Capítulo 6 — Bagging y Random Forest

6.1 Bagging (Bootstrap Aggregating)

Los árboles de decisión son modelos de **alta varianza**: pequeñas variaciones en los datos de entrenamiento pueden producir árboles completamente distintos. Bagging aborda esto promediando muchos árboles entrenados en versiones ligeramente diferentes de los datos.

Predictor Bagging

$$f_{\text{bag}}(x) = (1/M) \cdot \sum_{m=1}^M f_m(x)$$

$$\text{var}(f_{\text{bag}}(x)) = \sigma^2(x) / M \rightarrow 0 \text{ cuando } M \rightarrow \infty$$

Supuestos de Bagging: Cada estimador es i.i.d. (idéntica distribución, mutuamente independiente). Cuando se cumplen: $E[f_m(x)] = \mu$, $\text{var}[f_m(x)] = \sigma^2$, $\text{cov}(f_m, f_{m'}) = 0$. Entonces la varianza del promedio es σ^2/M , que tiende a 0 cuando $M \rightarrow \infty$.

6.1.1 Bootstrap — Remuestreo con Reemplazo

El **bootstrap** es la técnica que permite crear los M conjuntos de datos diferentes a partir del dataset original. Se seleccionan n muestras *con reemplazo* del dataset de n observaciones. Resultado: aproximadamente **63.2%** de observaciones únicas por muestra bootstrap (el 36.8% restante son duplicados).

6.1.2 Out-of-Bag (OOB) — Validación Sin Conjunto de Prueba

Una característica poderosa de Bagging: el ~36.8% de datos que NO se seleccionan en cada bootstrap (out-of-bag) sirven como conjunto de validación natural para ese árbol. El error OOB es un estimador prácticamente libre de sesgo del error de generalización, sin necesidad de separar un conjunto de validación adicional.

NOTA: El OOB score es un estimador casi tan bueno como la validación cruzada k-fold, pero computacionalmente mucho más eficiente ya que no requiere re-entrenar el modelo.

6.2 Random Forest (Bosque Aleatorio)

Bagging mejora la varianza, pero todos los árboles tienden a ser similares porque se entrenan con el mismo conjunto de características. Si existe una característica muy dominante, todos los árboles la usarán en el nodo raíz, y los árboles estarán **correlacionados entre sí**, lo que limita la reducción de varianza.

Random Forest soluciona esto introduciendo **aleatoriedad en la selección de características**: en cada nodo de cada árbol, se selecciona aleatoriamente un subconjunto de **m características** del total de **p**, y solo se busca la mejor división dentro de ese subconjunto.

Algoritmo Random Forest

Regresión: $f_{\text{rf}}(x) = (1/B) \cdot \sum_{b=1 \text{ to } B} f_b(x)$

Clasificación: $f_{\text{rf}}(x) = \text{argmax}(k \in y) \sum_{b=1 \text{ to } B} I(f_b(x)=k)$

Valores típicos: Clas. $m=\sqrt{p}$, $n_{\text{min}}=1$ | Regr. $m=p/3$, $n_{\text{min}}=5$

Hiperparámetros Clave de Random Forest

Hiperparámetro	Descripción	Efecto	Valor Típico
n_estimators	Número de árboles en el bosque	Más árboles → menos varianza (con rentas decrecientes)	100-500
max_features	Características consideradas por nodo	Menor m → menos correlación pero más sesgo	sqrt (reg.) p/3 (reg.)
max_depth	Profundidad máxima por árbol	Profundidad mayor → menos sesgo, más varianza	sin límite
min_samples_leaf	Mínimo de muestras por hoja	Valor mayor → regularización suave	1 (default)
ccp_alpha	Post-poda aplicada a cada árbol	Mayor alpha → árboles más simples	0 (sin poda)
oob_score	Usar OOB como métrica de validación	Permite crear conjunto de validación	True
n_jobs	Procesadores paralelos	-1 = todos los disponibles	-1

ADVERTENCIA: Un error común es aumentar n_estimators indefinidamente esperando mejoras lineales. Más de 200-500 árboles rara vez mejora significativamente el desempeño pero sí aumenta el tiempo de predicción.

Capítulo 7 — AdaBoost

7.1 Concepto e Historia

AdaBoost (**Adaptive Boosting**) fue propuesto por **Freund & Schapire en 1996** y recibió el premio Gödel (el equivalente al Nobel en Ciencias de la Computación Teórica). Fue el primer algoritmo de boosting práctico y exitoso, y demostró que podían combinarse clasificadores débiles (que solo aciertan ligeramente más del 50%) para formar uno fuerte.

Aspecto	Detalle
Origen	Freund & Schapire, 1996 (Premio Gödel)
Idea central	Repesar las muestras: las mal clasificadas reciben más peso en la siguiente iteración
Clasificador base	Normalmente un 'stump' (árbol de profundidad 1)
Predicción final	Votación ponderada: $\hat{y} = \text{sign}(\sum \alpha_i h_i(x))$
Función de pérdida	Minimiza la pérdida exponencial (interpretable como gradient descent aditivo)
Ventajas	Poco tuning necesario, buen rendimiento en datasets tabulares pequeños-medianos
Limitaciones	Sensible a ruido y clases desbalanceadas; no soporta valores nulos

7.2 Algoritmo AdaBoost Paso a Paso

El algoritmo puede resumirse en estos pasos iterativos para clasificación binaria:

Intuición del Algoritmo

Piensa en AdaBoost como un proceso de aprendizaje adaptativo similar a cómo aprende un estudiante:

- El primer clasificador intenta resolver el problema completo (con pesos iguales).
- Las muestras que falló se marcan como 'difíciles' y reciben más atención.
- El siguiente clasificador se especializa en esas muestras difíciles.
- Al final, el consenso ponderado de todos da una respuesta más robusta que cualquiera individualmente.

7.3 Implementación en scikit-learn

NOTA: AdaBoost sin sintonización de hiperparámetros ya da resultados mejores que un Random Forest con sintonización. Esto ilustra la potencia del boosting frente al bagging.

Capítulo 8 — Gradient Boosting

8.1 La Idea Central

Gradient Boosting reformula el boosting como un problema de **optimización de una función de pérdida** usando descenso por gradiente. En lugar de re-pesar muestras (AdaBoost), entrena cada nuevo árbol para predecir el **gradiente negativo de la función de pérdida** respecto a las predicciones actuales, es decir, los **residuos**.

Gradient Boosting — Predicción Final

$$F_M(x) = F_0(x) + \eta \cdot h_M(x)$$

$$F_M(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \dots + \eta \cdot h_M(x)$$

8.2 Algoritmo Gradient Boosting — Clasificación Binaria

8.3 Implementación en scikit-learn

Hiperparámetros Clave de Gradient Boosting

Hiperparámetro	Descripción	Impacto	Rango típico
n_estimators	Número de árboles (iteraciones)	Más → menos sesgo, riesgo overfitting	50-500
learning_rate (η)	Escala la contribución de cada árbol	Menor η → necesita más árboles, mejor generalización	0.01-0.3
max_depth	Prof. máxima por árbol individual	Menor → menos varianza. GB funciona bien con profundidad 3-5	3-10
min_samples_split	Min muestras para dividir un nodo	Mayor → regularización, menos overfitting	10-20
subsample	Fracción de datos por árbol (Stochastic GB)	Menor → más regularización, más lento converger	0.5-1.0
n_iter_no_change	Early stopping si no mejora	Evita sobreajuste por exceso de iteraciones	5-15
validation_fraction	Fracción usada para early stopping	Separación interna de validación	0.1

NOTA: La combinación *learning_rate* bajo + *n_estimators* alto suele dar mejores resultados pero requiere más tiempo de entrenamiento. Empieza con *lr*=0.1 y 100 árboles y ajusta según los resultados.

Capítulo 9 — Boosting Avanzado: XGBoost, LightGBM y CatBoost

9.1 XGBoost (eXtreme Gradient Boosting)

XGBoost mantiene la idea central de Gradient Boosting pero introduce mejoras algorítmicas que lo hacen más rápido, robusto y preciso. Es el algoritmo que domina competencias de Machine Learning en datos tabulares (Kaggle, etc.).

9.1.1 Mejoras Técnicas de XGBoost

Mejora	Problema que resuelve	Cómo funciona
Regularización explícita (γ, λ)	GB clásico puede sobreajustarse sin control de complejidad	Introduce penalizaciones extra. λ es regularización L2 sobre pesos de hojas. γ es penalización por número de hojas.
Newton Boosting (gradiente 2do orden)	GB clásico usa solo 1er orden (gradiente). Convergencia lenta	Usa el Hessiano (2da derivada). Converge más rápido. Permite funciones no lineales.
Búsqueda de splits con cuantiles ponderados	Buscar todos los valores posibles es $O(n \times p)$	Agrupa datos en bins ponderados por el Hessiano. Reduce a $O(\text{bins} \times p)$. Hace splits en cuantiles ponderados.
Manejo nativo de NaN	GB clásico requiere imputación previa	Aprende la dirección óptima (izq/der) para valores faltantes durante el entrenamiento.
Categorías nativas (enable_categorical)	One-Hot Encoding crea matrices dispersas y lentas	Optimiza categorías por estadísticas de gradiente. Partición óptima sin explosión de memoria.

9.1.2 Implementación XGBoost

9.2 LightGBM

LightGBM resuelve el problema de escala de XGBoost: en datasets con millones de filas o miles de características, la búsqueda de splits sigue siendo costosa. LightGBM introduce 4 mejoras algorítmicas fundamentales.

9.2.1 Las 4 Innovaciones de LightGBM

Innovación	Descripción	Beneficio
Histogramas (vs búsqueda exacta)	Discretiza cada feature en bins (ej: 255). Busca el mejor split en los límites de bins.	Quinientos millones de bins. Con 10M filas y 256 bins → 40000x más rápido.
Leaf-wise growth (vs level-wise)	Solo expande la hoja con mayor ganancia en cada paso.	Árbol asimétrico. Menor número de nodos, menor error. Riesgo de overfitting si no se controla.
GOSS (Gradient-based One-Side Sampling)	Solo usa muestras con gradiente alto (peor clasificadas).	Reduce datos por árbol de forma aleatoria (ej: 30%) sin perder precisión.
EFB (Exclusive Feature Bundling)	Fusiona features que nunca son nonzero simultáneamente.	Reduce número de features dispersos (one-hot encoding). Entrenamiento más eficiente.

Visualización: Level-wise vs Leaf-wise

9.2.2 Implementación LightGBM

9.3 CatBoost

CatBoost fue desarrollado por **Yandex** para resolver el problema de variables categóricas de alta cardinalidad sin necesidad de codificación previa. Su innovación principal es el tratamiento de categóricas sin introducir fuga de información.

9.3.1 Innovaciones de CatBoost

Innovación	Problema que resuelve	Cómo funciona
Ordered Target Statistics	Target encoding clásico filtra información de Pareto de features, target leakage	Para cada feature, se calcula la media del target usando SOLO las muestras de la clase de interés
Feature combinations	Interacciones entre categóricas no se capturan automáticamente	Se crean automáticamente nuevas features combinando múltiples categorías
Árboles simétricos	Predicción costosa con árboles asimétricos	La misma condición se aplica en todos los nodos de un nivel. Predicción

9.3.2 Implementación CatBoost

9.4 Comparativa de los 3 Algoritmos Avanzados

Criterio	XGBoost	LightGBM	CatBoost
Velocidad de entrenamiento	Media	Muy alta (histogramas+GOSS+EFB)	Media-alta
Datos grandes (>1M filas)	Lento	Excelente	Medio
Categorías	Requiere encoding manual	Soporte nativo básico	Soporte nativo avanzado (sin leakage)

Valores nulos	Sí (nativo)	Sí (nativo)	Solo en numéricas
Regularización	Explícita (γ , λ , α)	reg_alpha, reg_lambda	l2_leaf_reg
Early stopping	Sí (eval_set)	Sí (callbacks)	Sí (eval_set)
Crecimiento del árbol	Level-wise (configurable)	Leaf-wise (default)	Simétrico (level-wise)
Interpretabilidad	feature_importances_	feature_importances_ + split count	feature_importances_
Instalación	pip install xgboost	pip install lightgbm	pip install catboost
Versión (2026)	3.2.0	4.6.0	1.2.10
Mejor para...	Datos mixtos con tuning fino	Datasets muy grandes	Datos con muchas categóricas

Capítulo 10 — Conceptos Fundamentales

1. Sobreajuste (Overfitting)

El árbol memoriza el ruido del training set. Train acc=100%, test acc=77%. La causa: el árbol crece sin restricciones hasta que cada hoja tiene exactamente una muestra.

POR QUE ES CRITICO: SIEMPRE regularizar árboles. Sin regularización, un árbol de decisión es inútil para datos reales.

2. Entropía y Ganancia de Información

La entropía mide cuánta 'impureza' hay en un nodo. La ganancia mide cuánto reduce esa impureza al dividir. Se selecciona el atributo con mayor ganancia.

POR QUE ES CRITICO: Es el corazón del árbol: sin este criterio, el árbol no sabría qué pregunta hacer primero.

3. Impureza de Gini

Probabilidad de clasificar mal una muestra aleatoria. Más eficiente que entropía (sin logaritmos). Default en sklearn.

POR QUE ES CRITICO: En la práctica, entropía y Gini dan resultados casi idénticos. La diferencia de rendimiento es despreciable.

4. Podado (Pruning)

Reducir la complejidad del árbol. ccp_alpha busca el árbol óptimo tras construirlo. Pre-poda (max_depth, etc.) lo limita durante la construcción.

POR QUE ES CRITICO: La pre-poda es más eficiente; la post-poda con ccp_alpha es más sistemática y generalmente mejor.

5. Feature Importance

Los árboles (y ensambles) proveen importancia relativa de cada característica basada en cuánto contribuye a la reducción de impureza.

POR QUE ES CRITICO: No reemplaza análisis de causalidad, pero es una herramienta poderosa de selección de features.

6. Bootstrap (Remuestreo con reemplazo)

Técnica de muestreo que crea datasets sintéticos seleccionando n muestras con reemplazo. Cada bootstrap tiene ~63.2% de muestras únicas.

POR QUE ES CRITICO: Permite crear múltiples datasets para entrenar modelos distintos a partir de uno solo.

7. Out-of-Bag (OOB) Error

El ~36.8% de muestras no seleccionadas en cada bootstrap sirven como conjunto de validación gratuito. OOB error $\approx$ error de validación cruzada.

POR QUE ES CRITICO: Ahorra el costo computacional de una validación cruzada separada. Muy útil en datasets grandes.

8. Decorrelación (Random Forest)

RF añade aleatoriedad en la selección de features por nodo (m de p). Esto reduce la correlación entre árboles y amplifica el efecto del promedio.

POR QUE ES CRITICO: Sin decorrelación (Bagging puro), los árboles son similares y el promedio mejora poco la varianza.

9. Sesgo-Varianza Trade-off

Error = Sesgo² + Varianza + Ruido. Bagging reduce varianza manteniendo el sesgo. Boosting reduce sesgo pero puede aumentar varianza.

POR QUE ES CRITICO: Entender este tradeoff es esencial para decidir qué familia de algoritmo usar según el problema.

10. Learning Rate (η)

En boosting, escala la contribución de cada árbol. Valores pequeños requieren más árboles pero generalizan mejor. Hay un tradeoff η vs $n_{\text{estimators}}$.

POR QUE ES CRITICO: Regla práctica: $lr=0.1$ con 100 árboles como punto de partida, luego reducir lr y aumentar árboles para refinar.

11. Early Stopping

Detener el entrenamiento cuando el error de validación deja de mejorar. Previene overfitting sin necesidad de fijar $n_{\text{estimators}}$ a priori.

POR QUE ES CRITICO: Disponible en XGBoost, LightGBM y CatBoost. Ahorra tiempo y previene overfitting automáticamente.

12. Target Encoding

Reemplaza cada categoría con la media del target. Poderoso para alta cardinalidad pero susceptible a target leakage si no se maneja correctamente.

POR QUE ES CRITICO: Usar siempre TargetEncoder de sklearn (usa cross-fitting interno) o el enfoque ordenado de CatBoost.

13. Residuos en Gradient Boosting

Los residuos son el gradiente negativo de la función de pérdida. Cada árbol intenta 'corregir' los errores del conjunto anterior.

POR QUE ES CRITICO: La clave del éxito de GB: no intenta predecir y directamente, sino el ERROR actual del modelo.

14. Hessiano (Newton Boosting - XGBoost)

La segunda derivada de la función de pérdida. XGBoost la usa para una aproximación cuadrática más precisa, que converge más rápido.

POR QUE ES CRITICO: Permite a XGBoost converger con menos árboles que GB clásico para la misma precisión.

15. Stacking vs Averaging

Averaging = promedio simple de predicciones. Stacking = meta-modelo que aprende cómo combinar los modelos base. Stacking es más poderoso pero más costoso.

POR QUE ES CRITICO: En competencias Kaggle, stacking multi-nivel es la técnica que típicamente da el mejor resultado final.

Capítulo 11 — Relaciones Entre Conceptos

Reglas de Selección de Algoritmo

Situación	Algoritmo recomendado	Razón
Prototipo rápido + interpretabilidad	Árbol podado (max_depth=3-5)	Caja blanca, visualizable, reglas si-entonces
Dataset mediano, buena precisión	Random Forest	Robusto, pocos hiperparámetros, difícil de sobreajustar

Máxima precisión en datos tabulares	XGBoost / LightGBM / CatBoost	Estado del arte en Kaggle y producción
Dataset con muchas categorías	CatBoost	Manejo nativo sin leakage
Dataset con millones de filas	LightGBM	Histogramas + GOSS → mucho más rápido
Datos ruidosos / outliers	Gradient Boosting (lr bajo)	Menos sensible que AdaBoost al ruido
Clases muy desbalanceadas	XGBoost (scale_pos_weight)	Parámetro específico para desbalance
Datos sin NaN, pequeño-mediano	AdaBoost	Simple, efectivo, pocas decisiones de tuning

Capítulo 12 — Guía de Estudio Definitiva

12.1 Prerrequisitos

Prerrequisito	Nivel Requerido	Por qué es necesario
Python (pandas, numpy, matplotlib)	Intermedio	Todos los notebooks usan estas librerías extensamente
Estadística básica	Básico	Probabilidad, media, varianza, distribuciones
Álgebra lineal básica	Básico	Vectores, matrices (para entender feature importances)
Teoría de la información	Nociones	Entropía y ganancia de información
scikit-learn API	Básico-Intermedio	fit/predict/Pipeline/GridSearchCV/ColumnTransformer
Conceptos de ML	Básico	Overfitting, train/test split, cross-validation, métricas

12.2 Roadmap de Aprendizaje

12.3 Conceptos Difíciles y Cómo Estudiarlos

Concepto difícil	Error común	Estrategia para dominarlo
ccp_alpha	Confundirlo con un umbral de precisión	Visualizar el árbol con diferentes valores de alpha y observar cómo se simplifica
Residuos en GB	Creer que es la diferencia $y - \hat{y}$ directamente	Para MSE sí es así, pero para log-loss es el gradiente negativo de la pérdida
Learning rate vs n_estimators	Creer que son independientes	Son complementarios: Ir bajo requiere más árboles. Usar early stopping para
OOB error	Confundirlo con el error en test	OOB estima el error de generalización usando datos no vistos por ese árbol
Feature importance en RF	Creer que indica causalidad	Solo indica importancia relativa para la predicción. No implica causa-efecto.
Leaf-wise vs level-wise	No entender el riesgo de overfitting	Leaf-wise puede crear árboles muy desequilibrados. Controlar con num_leav
Target leakage en TE	Aplicar target encoding sin protección	Siempre usar sklearn TargetEncoder (cross-fitting) o CatBoost. Nunca calcula

12.4 Estrategia de Tuning Recomendada

Capítulo 13 — Cheatsheet & Fórmulas de Referencia

13.1 Fórmulas Esenciales

Concepto	Fórmula	Rango / Interpretación
Entropía	$H(S) = -\sum p(c) \cdot \log_2 p(c)$	$[0, \log_2(k)]$; 0=puro, max=mezcla uniforme
Ganancia información	$GI(S,a) = H(S) - \sum S_v / S \cdot H(S_v)$	$[0, H(S)]$; mayor=mejor split
Gini	$Gini(S) = 1 - \sum p_i^2$	$[0, 1-1/k]$; 0=puro
Gini split	$Gini_split = L /(L + R) \cdot Gini(L) + R /(L + R) \cdot Gini(R)$	Minimizar para seleccionar mejor split
Bagging	$\bar{f}_{bag}(x) = (1/M) \cdot \sum \bar{f}_m(x)$	Promedio simple de M modelos
Varianza bagging	$var(\bar{f}_{bag}) = \sigma^2/M \rightarrow 0$ cuando $M \rightarrow \infty$	Disminuye con más estimadores
RF predicción regr.	$\bar{f}_{rf}(x) = (1/B) \cdot \sum \bar{f}_b(x)$	Promedio de B árboles
RF predicción clas.	$\bar{f}_{rf}(x) = \operatorname{argmax}_k \sum I(\bar{f}_b(x)=k)$	Voto mayoritario de B árboles
AdaBoost error	$\epsilon = \sum w_i \cdot I(y_i \neq h(x_i)) / \sum w_i$	$[0, 0.5]$; <0.5=útil
AdaBoost alpha	$\alpha = \frac{1}{2} \cdot \ln((1-\epsilon)/\epsilon)$	Mayor $\alpha \rightarrow$ clasificador más influyente
AdaBoost weights	$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha \cdot y_i \cdot h(x_i))$	Aumenta peso de errores
AdaBoost predicción	$\hat{y} = \operatorname{sign}(\sum \alpha_i \cdot h_i(x))$	Votación ponderada
GB residuos	$r_i^{(t)} = -\partial L(y_i, F(x_i)) / \partial F(x_i)$	Gradiente negativo de pérdida
GB MSE residuos	$r_i^{(t)} = y_i - F^{(t)}(x_i)$	Diferencia directa para MSE
GB actualización	$F^{(t+1)}(x) = F^{(t)}(x) + \eta \cdot h^{(t+1)}(x)$	η =learning rate, $h^{(t+1)}$ =nuevo árbol
GB predicción final	$F_M(x) = F^{(0)}(x) + \sum \eta \cdot h^{(t)}(x)$	Suma acumulada de árboles
Sesgo-Varianza	$E_{total} = \text{Sesgo}^2 + \text{Varianza} + \text{Ruido}$	Tradeoff fundamental de ML

13.2 Comandos y Snippets de Referencia Rápida

Capítulo 14 — Ejemplos de Código Completos

14.1 Pipeline Completo de Clasificación con RF

14.2 Comparativa Completa de Modelos

14.3 Visualización de Árbol Podado con Reglas

Capítulo 15 — Conclusiones y Recomendaciones

15.1 Aprendizajes Principales

✓ Los árboles SIEMPRE sobreajustan sin regularización

Un árbol sin podado tiene $\text{accuracy}=100\%$ en train pero puede ser inútil en test. La regularización es obligatoria.

✓ Bagging reduce varianza, Boosting reduce sesgo

Son filosofías complementarias. Si el árbol individual ya tiene bajo sesgo (árbol profundo), Bagging es más útil. Si tiene alto sesgo (árbol poco profundo), Boosting es más útil.

✓ Random Forest: robusto y fácil de usar

Con muy poco tuning (solo `n_estimators` y `max_depth`) Random Forest ya da buenos resultados. Es el algoritmo 'seguro' cuando no se sabe qué usar.

✓ XGBoost/LightGBM/CatBoost son el estado del arte para datos tabulares

En virtualmente cualquier competencia con datos tabulares, uno de estos tres gana. En producción, LightGBM es favorito por velocidad.

✓ Los ensambles son cajas negras, pero con feature importance

Se pierde la interpretabilidad total del árbol, pero `feature_importances_` y SHAP values compensan parcialmente.

✓ El escalado NO ayuda a los árboles

Todos los modelos basados en árboles son invariantes al escalado. No desperdiciar tiempo en `StandardScaler` para estos modelos.

✓ Las categóricas requieren codificación (excepto CatBoost)

AdaBoost y GradientBoosting clásico NO soportan categóricas directamente. XGBoost y LightGBM sí con configuración. CatBoost es el mejor nativo.

✓ Los valores nulos no son problema para árboles

A diferencia de los modelos lineales, los árboles pueden manejar NaN de forma nativa (excepto AdaBoost).

15.2 Observaciones Críticas del Código

Observación	Archivo	Implicación
El notebook usa solo variables numéricas al entrenar el árbol de decisión.	Modelos_de_arboles_de_decision.ipynb	Se pierde el poder de las variables categóricas. Una mejora natural sería usar CatBoost o XGBoost.
AdaBoost se aplica solo tras dropna()	Modelos_de_boosting.ipynb	Correcto: AdaBoost no tolera NaN. CatBoost y XGBoost sí.
XGBoost usa grow_policy='lossguide'	Modelos_de_boosting_avanzados.ipynb	Equivale al leaf-wise de LightGBM. No es el default de XGBoost (level-wise).
LightGBM sin optimización ya supera RF Compacta notebooks	Modelos_de_boosting_avanzados.ipynb	Demuestra que GB avanzado tiene mejor baseline que bagging incluido.
La importancia de features varía entre algoritmos	Todos los notebooks	XGBoost prioriza 'relationship'; CatBoost prioriza 'capital-gain'. Ninguno prioriza 'age'.

15.3 Mejoras Posibles y Próximos Pasos

- Implementar **SHAP values** para interpretabilidad post-hoc de los ensambles (más informativo que feature_importances_).
- Usar **Optuna** en lugar de RandomizedSearchCV para búsqueda bayesiana de hiperparámetros (más eficiente).
- Implementar **stacking** de múltiples modelos (árbol + RF + XGBoost + LGBM como base, meta-modelo logístico).
- Explorar **calibración de probabilidades** (Platt scaling, isotonic regression) para modelos de boosting.
- Aplicar **análisis de curvas de aprendizaje** para diagnosticar si el problema es sesgo o varianza antes de elegir el algoritmo.
- Investigar **modelos de boosting con pérdidas personalizadas** (focal loss para clases muy desbalanceadas).

15.4 Tabla Resumen Final — Cuándo Usar Qué

Si necesitas...	Usa...	Parámetro clave
Interpretabilidad total + reglas	DecisionTree (max_depth=3-5)	ccp_alpha o max_depth
Baseline rápido y robusto	RandomForest (n_estimators=200)	max_features='sqrt'
Mejor precisión, datos medianos sin categorías	CatBoost o AdaBoost	learning_rate + n_estimators
Mejor precisión general + datos mixtos	XGBoost	gamma, reg_alpha, reg_lambda
Mejor precisión + datos grandes (>500K filas)	LightGBM	num_leaves, min_child_samples
Mejor precisión + muchas variables categóricas	CatBoost	cat_features, l2_leaf_reg
Máxima precisión (competencia)	Stacking de RF + XGB + LGBM + CAT	Meta-modelo Ridge/Logístico

Guía Maestra generada el 21 de Mayo de 2026

Emanuel Cabrera Novoa · Análisis Predictivo de Datos · Docente: Jhon Quiza